Project Title – Healthcare Analytics for Doctor Visits

[Sarthak Singh Tomar

Madhav Institute of Technology & Science]

AICTE STU ID :
(STU6a3910da1bf8f1782124762**)**

PROBLEM STATEMENT

- The objective of this project is to analyze doctor visits using Python and data analytics techniques. It examines the relationship between doctor visits and factors such as age, gender, income, illness, health status, insurance, and chronic conditions to identify healthcare utilization patterns through statistical analysis and data visualization.



Project Description

This project focuses on analyzing healthcare data to understand patterns in doctor visits using Python. It uses libraries such as Pandas, NumPy, Matplotlib, and Seaborn for data cleaning, statistical analysis, and visualization. The project examines how age, gender, income, illness, health status, insurance coverage, and chronic conditions are associated with doctor visits. The results help identify healthcare utilization patterns through data-driven insights.

WHO ARE THE END USERS?

- **Healthcare Providers:** Doctors and hospitals analyzing patient visit patterns.
- **Healthcare Administrators:** Planning and managing healthcare resources.
- **Researchers:** Studying factors affecting healthcare utilization.
- **Government and Policymakers:** Supporting healthcare planning and policy development.
- **Data Analysts:** Extracting insights from healthcare data.

Technology Used

- **Programming Language:** Python
- **Data Analysis:** Pandas, NumPy
- **Data Visualization:** Matplotlib, Seaborn
- **Development Environment:** Jupyter Notebook / Google Colab / VS Code
- **Dataset Format:** CSV (Comma-Separated Values)
- **Techniques:** Data Cleaning, Exploratory Data Analysis (EDA), Statistical Analysis, and Data Visualization.



RESULTS

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


df = pd.read_csv(r"C:\Users\sarth\Downloads\Telegram Desktop\1776250375-P2-Healthcare Analytics for Doctor Visits.csv"

)


if "Unnamed: 0" in df.columns:
    df.drop(columns=["Unnamed: 0"], inplace=True)

print("Dataset Shape:", df.shape)
print("\nColumn Names:\n", df.columns.tolist())

print("\nDataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Records:", df.duplicated().sum())

df.drop_duplicates(inplace=True)

print("\nCleaned Dataset:")
print(df.head())

df.to_csv("cleaned_healthcare_data.csv", index=False)

print("\nCleaned dataset saved successfully.")
```

```
Dataset Shape: (5190, 12)

Column Names:
 ['visits', 'gender', 'age', 'income', 'illness', 'reduced', 'health', 'private', 'freepoor', 'freerepat', 'nchronic', 'lchronic']

Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 5190 entries, 0 to 5189
Data columns (total 12 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   visits     5190 non-null   int64
 1   gender     5190 non-null   str
 2   age        5190 non-null   float64
 3   income     5190 non-null   float64
 4   illness    5190 non-null   int64
 5   reduced    5190 non-null   int64
 6   health     5190 non-null   int64
 7   private    5190 non-null   str
 8   freepoor   5190 non-null   str
 9   freerepat  5190 non-null   str
 10  nchronic   5190 non-null   str
 11  lchronic   5190 non-null   str
dtypes: float64(2), int64(4), str(6)
memory usage: 486.7 KB
...
3        no       no       no
4        no      yes       no

Cleaned dataset saved successfully.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

https://github.com/SarthakTomar2008

RESULTS

```python
print("Descriptive Statistical Analysis:")
print(df.describe())

print("\nAverage Doctor Visits:", df["visits"].mean())

print("Median Doctor Visits:", df["visits"].median())

print("Maximum Doctor Visits:", df["visits"].max())

print("Minimum Doctor Visits:", df["visits"].min())

print("Average Age:", df["age"].mean())

print("Average Income:", df["income"].mean())

print("Average Illness:", df["illness"].mean())

print("Average Health Score:", df["health"].mean())
```

```
Descriptive Statistical Analysis:
            visits          age       income      illness      reduced  \
count  3870.000000  3870.000000  3870.000000  3870.000000  3870.000000
mean      0.389147     0.410762     0.582550     1.670543     1.151680
std       0.899433     0.200034     0.377685     1.421045     3.286601
min       0.000000     0.190000     0.000000     0.000000     0.000000
25%       0.000000     0.220000     0.250000     1.000000     0.000000
50%       0.000000     0.320000     0.550000     1.000000     0.000000
75%       1.000000     0.620000     0.900000     2.750000     0.000000
max       9.000000     0.720000     1.500000     5.000000    14.000000

            health
count  3870.000000
mean      1.577778
std       2.330537
min       0.000000
25%       0.000000
50%       1.000000
75%       2.000000
max      12.000000

Average Doctor Visits: 0.3891472868217054
Median Doctor Visits: 0.0
Maximum Doctor Visits: 9
Minimum Doctor Visits: 0
Average Age: 0.4107622739018088
Average Income: 0.5825503875968993
Average Illness: 1.6705426356589148
Average Health Score: 1.5777777777777777
```

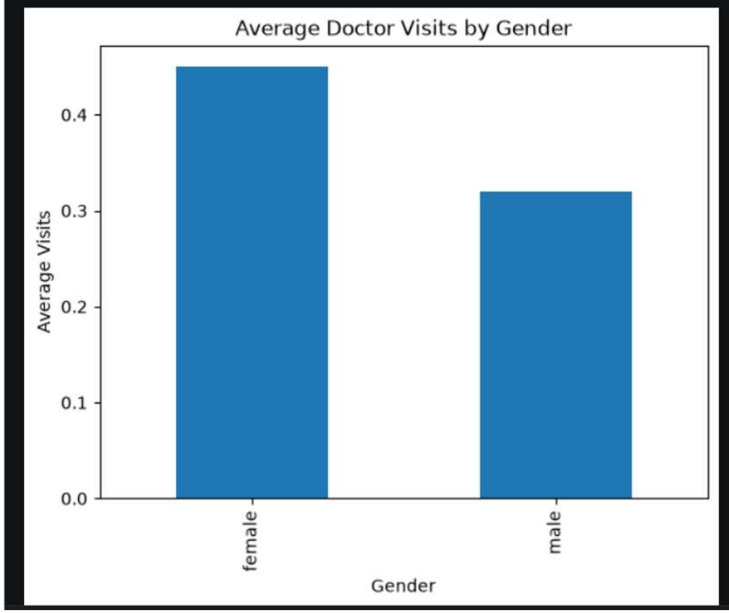
https://github.com/SarthakTomar2008

RESULTS

```
print(df.groupby("gender")["visits"].mean())
print(df["gender"].value_counts())

df.groupby("gender")["visits"].mean().plot(kind="bar")
plt.title("Average Doctor Visits by Gender")
plt.xlabel("Gender")
plt.ylabel("Average Visits")
plt.show()
```

```
gender
female    0.449710
male      0.319645
Name: visits, dtype: float64
gender
female    2068
male      1802
Name: count, dtype: int64
```



https://github.com/SarthakTomar2008

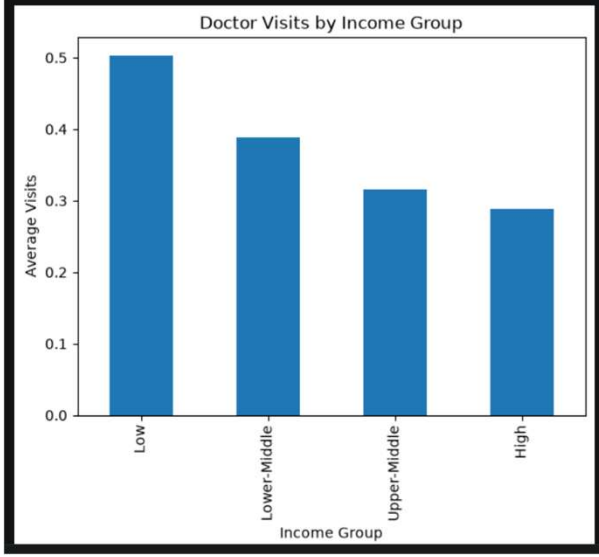
RESULTS

```
Click to add a breakpoint = pd.qcut(
    df["income"],
    q=4,
    labels=["Low", "Lower-Middle", "Upper-Middle", "High"],
    duplicates="drop"
)

print(df.groupby("income_group", observed=False)["visits"].mean())

df.groupby("income_group", observed=False)["visits"].mean().plot(kind="bar")
plt.title("Doctor Visits by Income Group")
plt.xlabel("Income Group")
plt.ylabel("Average Visits")
plt.show()
```

```
income_group
Low             0.503339
Lower-Middle    0.388190
Upper-Middle    0.316135
High            0.287958
Name: visits, dtype: float64
```



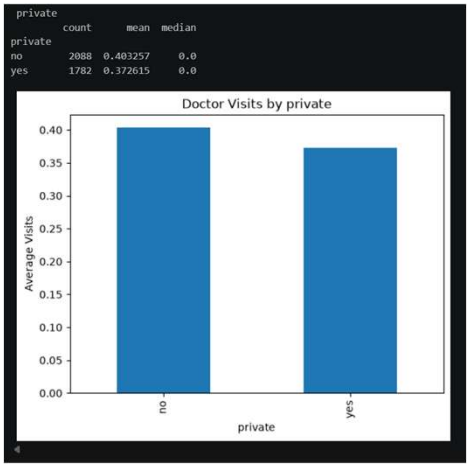
https://github.com/SarthakTomar2008

RESULTS

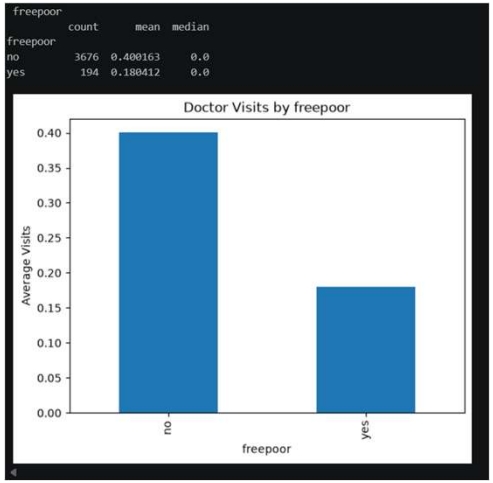
```
for col in ["private", "freepoor", "freerepat"]:
    print("\n", col)
    print(df.groupby(col)["visits"].agg(["count", "mean", "median"]))

    df.groupby(col)["visits"].mean().plot(kind="bar")
    plt.title("Doctor Visits by " + col)
    plt.xlabel(col)
    plt.ylabel("Average Visits")
    plt.show()
```

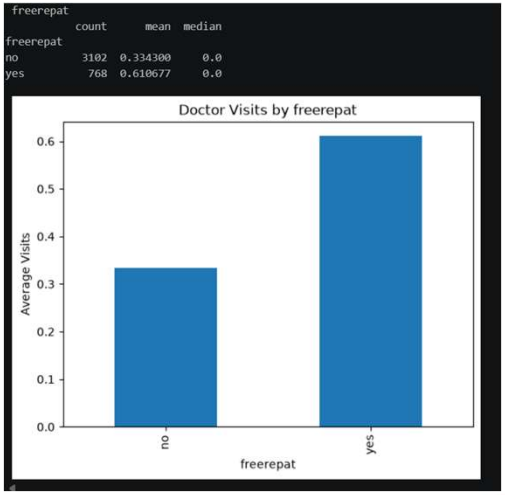
```
private
         count      mean  median
private
no        2088  0.403257     0.0
yes       1782  0.372615     0.0
```




```
freepoor
          count      mean  median
freepoor
no         3676  0.400163     0.0
yes         194  0.180412     0.0
```




```
freerepat
          count      mean  median
freerepat
no         3102  0.334300     0.0
yes         768  0.610677     0.0
```



https://github.com/SarthakTomar2008

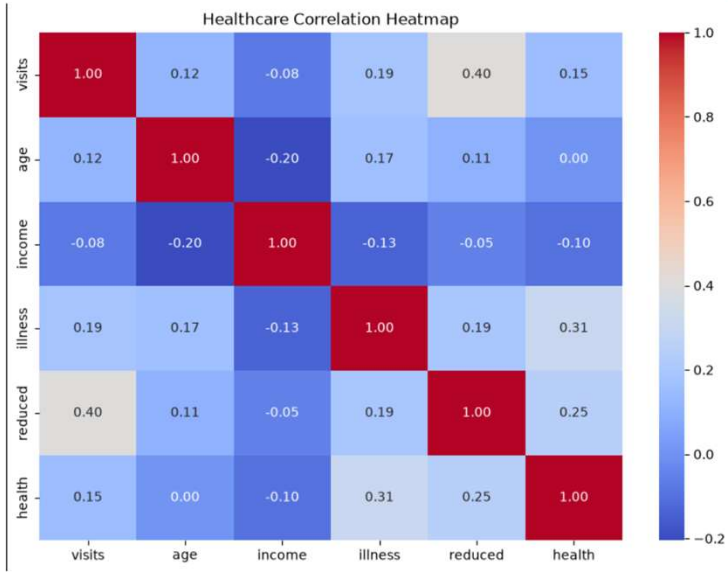
RESULTS

```
correlation_matrix = df.select_dtypes(include="number").corr()

print(correlation_matrix)

plt.figure(figsize=(10, 7))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Healthcare Correlation Heatmap")
plt.show()
```

	visits	age	income	illness	reduced	health
visits	1.000000	0.123836	-0.076916	0.185469	0.403213	0.149304
age	0.123836	1.000000	-0.203979	0.167224	0.105057	0.003812
income	-0.076916	-0.203979	1.000000	-0.134864	-0.053131	-0.096036
illness	0.185469	0.167224	-0.134864	1.000000	0.191833	0.309573
reduced	0.403213	0.105057	-0.053131	0.191833	1.000000	0.246496
health	0.149304	0.003812	-0.096036	0.309573	0.246496	1.000000



AND MORE….

https://github.com/SarthakTomar2008

Thank you